SyncPass

Informe técnico y plan de verificación | Ethan Astorga | 12 de noviembre de 2026

Alcance: versión del repositorio local **main**, compilaciones y pruebas ejecutadas hoy. El estado de AWS y la prueba física se indican por separado. Este documento no presenta como terminadas las funciones que todavía no tienen evidencia.

1. Problema, solución y arquitectura

La operación de talleres comunitarios necesita registrar catálogo, asistencia, encuestas y quórum sin depender de planillas dispersas. SyncPass ofrece un mismo backend para una web administrativa y una app Android de terreno. Los datos compartidos se guardan en PostgreSQL; cada cliente conserva datos locales para señal intermitente.

Web React/Vite en Vercel + Android Kotlin/Compose -> AWS API Gateway -> Caddy + Spring Boot JAR en EC2/VPC -> Supabase PostgreSQL. La API usa Amazon Cognito para identidad y grupos. Web guarda borradores en IndexedDB; Android usa Room/SQLite. Los clientes publican y consultan a través de la API.

Componente	Razón técnica	Dónde comprobarlo
Vercel + React/Vite/TypeScript	Entrega estática HTTPS, componentes y compilación rápida. Wouter resuelve rutas; CSS propio y Tailwind aportan diseño.	web-react/package.json; src/main.tsx; src/pages; vite.config.js
API Gateway + EC2	Una URL HTTPS de entrada y un JAR Java separado del frontend. Caddy termina TLS y reenvía a 127.0.0.1:8080.	deploy/Caddyfile; deploy/update-lab.sh; docs/OPERACION_LAB.md
Cognito + Spring Security	Cognito gestiona usuarios/grupos; el backend valida JWT y autoriza rutas.	backend/AuthController.java; SecurityConfig.java; controladores
Supabase PostgreSQL + Flyway	Base compartida con migraciones versionadas y acceso JPA.	backend/src/main/resources/db/migration; entidades/repositorios
Room + IndexedDB	Persistencia y cola local por dispositivo para trabajo intermitente.	mobile-kotlin/.../SyncPassRepository.kt; web-react/src/data/localDb.js

2. Capas y recorrido de datos

En Android, Compose es la vista; **HomeViewModel** expone estado y acciones; **SyncPassRepository** coordina Retrofit remoto y Room local. Los DTO describen JSON y algunas entidades Room. En web, páginas/componentes React mantienen estado y usan **clients/api.ts** para HTTP e **IndexedDB** para persistencia local; no existe aún una capa única de repositorio web. En backend, controladores REST procesan JSON, repositorios JPA consultan el modelo y Spring Security aplica permisos. Esto es MVC para la API, no MVC puro para Android.

Ejemplo verificable: un admin crea un taller en la web mediante POST /api/eventos, que pasa por Gateway y EC2 hasta PostgreSQL. Android actualiza catálogo con GET /api/eventos y guarda la respuesta en Room. Una asistencia móvil pendiente entra a Room, luego POST /api/sync/batch; la API responde acceptedIds y la web consulta GET /api/quorum. Solo después de este recorrido se puede afirmar que ambos clientes comparten datos.

3. Funciones y contratos principales

Función	Ruta/cliente	Persistencia y permiso
Registro, confirmación, login	/api/auth/register, confirm, login, refresh, logout	Cognito a través de EC2; registro nuevo debe iniciar como participante.
Proyectos: CRUD	/api/proyectos; web /proyectos	JPA/PostgreSQL; admin o coordinador.
Talleres: CRUD	/api/eventos; web /talleres; Android catálogo	JPA/PostgreSQL; lectura pública, escritura admin/coordinador.
Asistencia y encuesta	POST /api/sync/batch	Cola IndexedDB o Room; backend acepta IDs y evita borrar pendientes rechazados.
Quórum	GET /api/quorum; web /panel	Backend; admin, coordinador o monitor.
QR y GPS	QR SP1:UUID; ubicación en sincronización	La API comprueba identificador, radio y precisión cuando se configura geocerca.

Roles de Cognito: admin, coordinador, monitor y participante. Ocultar una opción en React o Compose mejora la interfaz; la autorización efectiva debe confirmarse con respuestas 401/403/200 del backend. API Gateway no debe describirse como el componente que decide roles si esa política está en Spring.

4. Ejecución y despliegue

• Web: entrar a web-react, configurar VITE_API_URL fuera de Git, ejecutar npm ci, npm run build y npm run coverage. Vercel publica el resultado compilado.

• Backend: JDK compatible, variables DATABASE_*, COGNITO_* y CORS_ORIGINS fuera de Git; ejecutar mvnw.cmd test y mvnw.cmd package. EC2 corre el JAR con systemd tras Caddy; Flyway migra al iniciar.

• Android: abrir mobile-kotlin en Android Studio o ejecutar gradlew.bat :app:assembleDebug. El APK debug se crea en app/build/outputs/apk/debug. Su API_BASE_URL apunta a Gateway; instalarlo en teléfono y conceder permisos de cámara/ubicación al probar.

• Tras reiniciar el laboratorio, verificar EC2, syncpass-api, caddy y syncpass-gateway-sync. La IP puede cambiar; el procedimiento de docs/OPERACION_LAB.md actualiza nip.io y la integración. No publicar secretos, claves ni contraseñas.

5. Evidencia del 12 de noviembre de 2026

Comprobación	Resultado	Interpretación
Vercel portada	HTTP 200	La web sirve contenido; no prueba CRUD ni actualización.
Gateway health	HTTP 503	Bloquea hoy la prueba web-API-móvil. Revisar laboratorio/EC2/Caddy/JAR/integración.
Vitest web	37 pruebas pasan; 93,35% líneas, 85,44% ramas	Incluye bitácora, pero no mide toda la web.
Maven backend	18 pruebas, 0 fallos, 0 errores	Incluye bitácora; H2 local no prueba Supabase desplegado.
Android Gradle	assembleDebug correcto	APK compila; sin teléfono ADB para ejecución física hoy.
Bitácora consultable	API, web y Android compilan localmente	Resumen extractivo y preguntas con cita; falta despliegue y prueba cruzada.

6. Plan de aceptación por puertas

A. Infraestructura

Gateway health y catálogo 200; sin token 401, rol insuficiente 403, admin 200. Registrar commit del JAR, IP y servicios. Resolver primero el 503.

B. Web y permisos

Registro nuevo → participante; repetir login; cuatro menús; CRUD completo de proyectos y talleres, con IDs, respuestas, persistencia tras refrescar y rechazo sin permiso.

C. Comunicación web-móvil

Crear taller web, verlo por ID en Android, probar Room sin red, editar web y actualizar móvil. Acreditar QR, guardar pendiente offline, sincronizar acceptedIds y ver quórum en web. Probar reintento, encuesta y GPS dentro/fuera de radio.

D. Calidad y diseño

Conservar reportes de pruebas; instrumentar toda la web antes de proclamar 80% global. Añadir pruebas Android de Room/Retrofit y contratos JSON. Comparar pantallas web/móvil y accesibilidad.

7. Innovación implementada localmente: bitácora consultable

Un administrador o coordinador publica apuntes o dicta texto en la web con consentimiento. Spring Boot guarda una bitácora por taller, selecciona hasta tres fragmentos para un resumen extractivo y responde preguntas con un fragmento citado. Android consulta el mismo taller y permite preguntar. El algoritmo compara términos normalizados; no usa un modelo generativo ni inventa respuestas cuando no encuentra coincidencia. Flyway V4 crea la tabla en PostgreSQL. Archivos: backend/BitacoraController.java, web-react/src/pages/BitacoraPage.jsx y mobile-kotlin/.../BitacoraScreen.kt.

La web ofrece dictado y grabación descargable en el dispositivo. SyncPass solo persiste texto: el audio no se sube a la API. No hay transcripción automática del archivo, grabación Android ni caché Room de esta bitácora. El MVP requiere conexión y todavía no se ha desplegado ni comprobado entre web y teléfono. Ver docs/INNOVACION_BITACORA.md para permisos, contrato REST y prueba de aceptación.

8. Guion de demostración y criterio de terminado

Mostrar mockups y problema, luego arquitectura y dos fuentes de datos, registro/login, dos mantenedores web, el mismo taller en Android, QR y cola offline, sincronización y quórum, respuestas 401/403/200, pruebas y límites. Conservar capturas con hora, versión e IDs ficticios; nunca capturar tokens o contraseñas. El sistema queda listo cuando las cuatro puertas del plan tienen evidencia y Gateway deja de responder 503.

9. Matriz de pruebas cruzadas que falta completar

ID	Acción	Resultado exigido
P01	Crear proyecto y taller en web; recargar	IDs y datos conservados en GET, incluso tras nueva sesión.
P02	Abrir catálogo en Android conectado y luego sin red	Mismo ID remoto y copia Room visible sin conexión.
P03	Editar taller en web y actualizar Android	Nombre, fecha y cupos coinciden; no queda una copia antigua.
P04	Guardar asistencia móvil sin red; reconectar	Pendiente Room pasa a acceptedIds; quórum web aumenta una sola vez.
P05	Escanear QR válido e inválido	Válido se acredita; inválido se rechaza con mensaje claro.
P06	Probar GPS dentro y fuera del radio	Dentro se acepta; fuera o sin permiso se rechaza según regla de API.
P07	Probar cuatro perfiles y token ausente	Menú correcto y API 200/403/401 según rol, sin fuga de datos.
P08	Registrar usuario nuevo y cambiar rol	Empieza participante; nuevo grupo solo tras cambio autorizado y nueva sesión.
P09	Publicar bitácora web y consultar Android	Mismo tallerId, resumen y fragmento citado; pregunta ausente recibe rechazo explícito.

10. Diagnóstico cuando una parte falla

Síntoma	Comprobar en orden
Web abre, CRUD falla	Consola de red; VITE_API_URL compilada; Gateway health; CORS; token y expiración.
Gateway 503	Laboratorio activo, IP actual, integración Gateway, DNS nip.io, certificado Caddy, servicio JAR y puerto 8080 local.
Login falla	Respuesta de /api/auth/login, salud de EC2, configuración de cliente Cognito, estado de usuario y grupo; nunca registrar clave.
Móvil no ve taller	API_BASE_URL del APK, red del teléfono, GET /api/eventos, refresco Room y comparación por ID.
Pendiente no se elimina	Respuesta acceptedIds, mismo owner, lote <=100, token vigente y error del backend; evitar borrar registros no aceptados.
Quórum no cambia	Asistencia aceptada en base, tallerId correcto, evitar doble conteo por reintento, refrescar GET /api/quorum.

11. Decisiones y límites declarados

• El laboratorio AWS Academy no garantiza disponibilidad permanente. Vercel puede seguir en línea mientras el backend está caído.

• El APK debug es apto para prueba académica, no una publicación Play Store. Su sesión se conserva en memoria; cerrar el proceso obliga a iniciar sesión de nuevo.

• La sincronización automática depende de que la app esté abierta; todavía no usa WorkManager. La geocerca y el QR requieren validación física en la versión final.

• La cifra Vitest observada corresponde al conjunto instrumentado. Para exigir 80% global hay que ampliar cobertura a las páginas y flujos restantes y publicar ese nuevo reporte.

• El MVP de bitácora es extractivo y solo se ha probado localmente. Audio persistente y transcripción Android necesitan consentimiento, almacenamiento, retención, costo y pruebas de exactitud antes de exponerlos.

Detalle operativo y casos paso a paso: docs/PLAN_VERIFICACION_COMUNICACION.md. Código y documentación: repositorio ethan1213/SyncPass.